Web Application Security Audit Report

Target Application: Kitchenware & Lunchbox E-commerce Platform
Prepared By: Dharmik Niteenbhai Poshiya (Cyber Security Intern)
Date: 17 October 2026

1. Introduction:

This document presents the findings of a Web Application Security Review conducted on the "Kitchenware & Lunchbox E-commerce Platform". The primary objective of this assessment is to proactively identify potential security vulnerabilities within the application's architecture before its public launch.

The review focuses on analyzing the application against common web vulnerabilities, specifically mapping identified risks to the OWASP Top 10 security framework. By identifying these weaknesses conceptually, this report aims to provide actionable secure development practices to the engineering team, ensuring the protection of sensitive customer data, secure transactions, overall platform integrity.

2. Scope of Assessment:

The scope of this security review is strictly limited to the application layer of the e-commerce platform. The assessment evaluates how user inputs and sessions are handled across key features.

Target Architecture: React (TypeScript) for the Frontend, Laravel for the Backend, and MySQL for the Database.

A. In-Scope Features for Analysis:

1. User Authentication: Login, Registration, and Session Management.

2. User Input & Forms: Product Search bar and Product Review/Comment sections.

3. File Management: User profile picture upload functionality.

4. Business Logic: Shopping cart and checkout workflows.

B. Out-of-Scope:

1. Active exploitation or testing on live production systems.

2. Network-level vulnerability scanning or Denial of Service (DoS/DDoS) testing.

3. Social Engineering or Physical Security assessments.

Methodology: Conceptual security analysis and threat modeling based on the OWASP Top 10 standard.

3. Identified Risks & Vulnerabilities:

Based on the architectural review of the Kitchenware & Lunchbox E-commerce Platform, the following conceptual vulnerabilities have been identified and mapped to the OWASP Top 10 framework.

1. SQL Injection (SQLi)

A. Location: Product Search Bar & User Login Form.

B. OWASP Mapping: A03:2021 - Injection.

C. Vulnerability Explanation: If the Laravel backend accepts user input from the React frontend (e.g., searching for a "Steel Lunchbox") and concatenates it

directly into a raw database query without parameterization or sanitation, an attacker can manipulate the query structure.

D. Potential Impact: An attacker could bypass authentication, read sensitive customer data (like passwords and addresses), or drop the database entirely.

E. Code Context (Conceptual): Vulnerable Approach: $users = DB::select("SELECT * FROM users WHERE email = '" . $_POST['email'] . "'");

F. Secure Approach: Using Laravel's built-in Eloquent ORM or prepared statements (e.g.,User::where('email', $request->email)->first();).

2. Stored Cross-Site Scripting (XSS)

1. Location: Product Review & Comment Section.

2. OWASP Mapping: A03:2021 - Injection (historically XSS).

3. Vulnerability Explanation: Users can submit reviews for kitchenware products. If the backend saves a review containing malicious JavaScript (e.g., <script>...</script>) without encoding it, and the React frontend renders this data dangerously (e.g., using dangerouslySetInnerHTML), the script will execute in the browsers of all users viewing that product.

4. Potential Impact: Attackers can steal session cookies, hijack user accounts, or redirect customers to fraudulent payment gateways.

5. Prevention Strategy: Sanitize all input on the Laravel backend before saving, and rely on React's default behavior of escaping text during rendering.

3. Insecure File Upload (Remote Code Execution Risk)

1. Location: User Profile Picture Upload Feature.

2. OWASP Mapping: A04:2021 - Insecure Design / A01:2021 - Broken Access Control.

3. Vulnerability Explanation: The application allows users to upload avatars. If the backend only checks the file extension on the frontend, an attacker can bypass this by intercepting the request and uploading a malicious executable file (e.g., a shell.php file disguised as an image).

4. Potential Impact: If the server executes the uploaded .php file, the attacker gains Remote Code Execution (RCE), leading to full server compromise and access to the backend infrastructure.

4. Recommended Security Measures (Secure Development Practices):

To mitigate the identified risks, the development team must implement the following secure coding practices:

1. Enforce Strict Input Validation & Parameterization: Never trust user input. Use Laravel's Eloquent ORM for all database interactions to automatically utilize prepared statements and prevent SQL Injection.

2. Implement Context-Aware Output Encoding: Ensure all user-generated content (like product reviews) is properly encoded before rendering in the React frontend to prevent XSS. Avoid dangerouslySetInnerHTML.

3. Secure File Upload Mechanisms:

A. Validate file types on the server side using MIME-type verification, not just file extensions.

B. Store uploaded files in a restricted, non-executable directory (e.g., AWS S3 or a separate storage disk).

C. Rename files randomly upon upload to prevent predictable file paths.

4. Session Management: Enforce secure, HttpOnly, and SameSite flags on all session cookies to prevent cookie theft via client-side scripts.

5. Conclusion:

The Kitchenware & Lunchbox E-commerce Platform possesses foundational e-commerce capabilities, but without robust security controls, it is susceptible to critical vulnerabilities like SQLi, XSS, and RCE. By integrating the recommended secure development practices into the CI/CD pipeline and adhering to the OWASP Top 10 guidelines, the organization can significantly reduce its attack surface and ensure a safe shopping experience for its customers.